Experiment No. 10

Experiment Name - End-to-End Data Analytics Project: Business Intelligence and Predictive Analytics using a Real-World Dataset

Steps

NAME : Atul Krishna

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Mastery in Data Analytics and Visualizations and Career Advancement

ROLL NO. : 14

CU ID : CU24250231

1. Select a real-world dataset relevant to a business or social problem, such as customer churn prediction, sales forecasting,employee attrition analysis, or house price prediction.

```
# Selecting a Real-World Dataset
from google.colab import files
uploaded = files.upload()

import os
print(os.listdir())

dataset_name = "House Price Prediction Dataset"

print("Selected Dataset:", dataset_name)
print("Dataset selected successfully!")
```

```
Choose Files House Pric... Dataset.csv
House Price Prediction Dataset.csv (text/csv) - 91297 bytes, last modified: 9/6/2026 - 100% done
Saving House Price Prediction Dataset.csv to House Price Prediction Dataset.csv
['.config', 'House Price Prediction Dataset.csv', 'sample_data']
Selected Dataset: House Price Prediction Dataset
Dataset selected successfully!
```

2. Collect and import the dataset, then perform data preprocessing by handling missing values, duplicates, outliers, andcategorical variables.

```
import pandas as pd
import numpy as np
import os

# Find the uploaded CSV file
csv_files = [f for f in os.listdir() if f.lower().endswith(".csv")]

# Use the uploaded House Price dataset
matching_files = [
    f for f in csv_files
    if "house price prediction dataset" in f.lower()
]

file_name = matching_files[0] if matching_files else csv_files[0]

df = pd.read_csv(file_name, encoding="latin1")

print("Dataset loaded successfully!")
print("File Name:", file_name)
print("Dataset Shape:", df.shape)

print("\nFirst 5 Rows:")
display(df.head())

print("\nDataset Information:")
df.info()

print("\nColumn Names:")
print(df.columns.tolist())
```

```
Dataset loaded successfully!
File Name: House Price Prediction Dataset.csv
Dataset Shape: (2000, 10)

First 5 Rows:
```

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location	Condition	Garage	Price
0	1	1360	5	4	3	1970	Downtown	Excellent	No	149919
1	2	4272	5	4	3	1958	Downtown	Excellent	No	424998
2	3	3592	2	2	3	1938	Downtown	Good	No	266746
3	4	966	4	2	2	1902	Suburban	Fair	Yes	244020
4	5	4926	1	4	2	1975	Downtown	Fair	Yes	636056

```
Dataset Information:
<class 'pandas.core.frame.DataFrame' >
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 10 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Id         2000 non-null   int64
 1   Area       2000 non-null   int64
 2   Bedrooms   2000 non-null   int64
 3   Bathrooms  2000 non-null   int64
 4   Floors     2000 non-null   int64
 5   YearBuilt  2000 non-null   int64
 6   Location   2000 non-null   object
 7   Condition  2000 non-null   object
 8   Garage     2000 non-null   object
 9   Price      2000 non-null   int64
dtypes: int64(7), object(3)
memory usage: 156.4+ KB

Column Names:
['Id', 'Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt', 'Location', 'Condition', 'Garage', 'Price'   ]
```

```
# Check Missing Values
print("Missing Values in Each Column:" )
print(df.isnull().sum())

print("\nTotal Missing Values:" , df.isnull().sum().sum())

# Check Duplicate Rows
print("\nDuplicate Rows:", df.duplicated().sum())

# Check Data Types
print("\nData Types:" )
print(df.dtypes)

# Statistical Summary
print("\nStatistical Summary:" )
display(df.describe(include="all").T)
```

```
Missing Values in Each Column:
Id           0
Area         0
Bedrooms     0
Bathrooms    0
Floors       0
YearBuilt    0
Location     0
Condition    0
Garage       0
Price        0
dtype: int64

Total Missing Values: 0

Duplicate Rows: 0

Data Types:
Id            int64
Area          int64
Bedrooms      int64
Bathrooms     int64
Floors        int64
YearBuilt     int64
Location     object
Condition    object
Garage       object
Price         int64
dtype: object

Statistical Summary:
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
Id	2000.0	NaN	NaN	NaN	1000.5	577.494589	1.0	500.75	1000.5	1500.25	2000.0
Area	2000.0	NaN	NaN	NaN	2786.2095	1295.146799	501.0	1653.0	2833.0	3887.5	4999.0
Bedrooms	2000.0	NaN	NaN	NaN	3.0035	1.424606	1.0	2.0	3.0	4.0	5.0
Bathrooms	2000.0	NaN	NaN	NaN	2.5525	1.10899	1.0	2.0	3.0	4.0	4.0
Floors	2000.0	NaN	NaN	NaN	1.9935	0.809188	1.0	1.0	2.0	3.0	3.0
YearBuilt	2000.0	NaN	NaN	NaN	1961.446	35.926695	1900.0	1930.0	1961.0	1993.0	2023.0
Location	2000	4	Downtown	558	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Condition	2000	4	Fair	521	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Garage	2000	2	No	1038	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Price	2000.0	NaN	NaN	NaN	537676.855	276428.845719	50005.0	300098.0	539254.0	780086.0	999656.0

```python
# Create a copy of the original dataset
df_clean = df.copy()

# Standardize column names
df_clean.columns = df_clean.columns.str.strip()

# Remove duplicate rows
df_clean = df_clean.drop_duplicates()

# Remove rows with missing target values
df_clean = df_clean.dropna(subset=["Price"])

# Convert numeric columns into numeric format
numeric_columns = [
    "Id", "Area", "Bedrooms", "Bathrooms",
    "Floors", "YearBuilt", "Price"
]

for col in numeric_columns:
    if col in df_clean.columns:
        df_clean[col] = pd.to_numeric(
            df_clean[col], errors="coerce"
        )

# Standardize categorical columns
categorical_columns = ["Location", "Condition", "Garage"]

for col in categorical_columns:
    if col in df_clean.columns:
        df_clean[col] = df_clean[col].apply(
            lambda x: x.strip() if isinstance(x, str) else x
        )

# Remove rows with invalid target values
df_clean = df_clean.dropna(subset=["Price"])
df_clean = df_clean[df_clean["Price"] > 0]

# Remove rows with invalid numeric feature values
# Missing feature values will be handled later by the ML pipeline.
if "Area" in df_clean.columns:
    df_clean = df_clean[
        df_clean["Area"].isna() | (df_clean["Area"] > 0)
    ]
```

```
df_clean = df_clean.reset_index(drop=True)

print("Data Preprocessing Completed!")
print("Original Dataset Shape:", df.shape)
print("Cleaned Dataset Shape:", df_clean.shape)

print("\nMissing Values After Cleaning:")
print(df_clean.isnull().sum())

print("\nDuplicate Rows After Cleaning:", df_clean.duplicated().sum())

display(df_clean.head())
```

```
Data Preprocessing Completed!
Original Dataset Shape: (2000, 10)
Cleaned Dataset Shape: (2000, 10)

Missing Values After Cleaning:
Id            0
Area          0
Bedrooms      0
Bathrooms     0
Floors        0
YearBuilt     0
Location      0
Condition     0
Garage        0
Price         0
dtype: int64

Duplicate Rows After Cleaning: 0
```

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location	Condition	Garage	Price
0	1	1360	5	4	3	1970	Downtown	Excellent	No	149919
1	2	4272	5	4	3	1958	Downtown	Excellent	No	424998
2	3	3592	2	2	3	1938	Downtown	Good	No	266746
3	4	966	4	2	2	1902	Suburban	Fair	Yes	244020
4	5	4926	1	4	2	1975	Downtown	Fair	Yes	636056

```
# Detect Outliers Using the IQR Method
outlier_columns = [
    "Area", "Bedrooms", "Bathrooms",
    "Floors", "YearBuilt", "Price"
]

outlier_results = []

for col in outlier_columns:
    values = df_clean[col].dropna()

    Q1 = values.quantile(0.25)
    Q3 = values.quantile(0.75)
    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    outlier_count = (
        (values < lower_limit) | (values > upper_limit)
    ).sum()

    outlier_results.append({
        "Column": col,
        "Lower Limit": lower_limit,
        "Upper Limit": upper_limit,
        "Outlier Count": int(outlier_count)
    })

outlier_df = pd.DataFrame(outlier_results)

print("Outlier Detection Results:")
display(outlier_df)

print("Outlier detection completed. Potential outliers are retained.")
```

```
Outlier Detection Results:
```

	Column	Lower Limit	Upper Limit	Outlier Count
0	Area	-1698.75	7239.25	0
1	Bedrooms	-1.00	7.00	0
2	Bathrooms	-1.00	7.00	0
3	Floors	-2.00	6.00	0
4	YearBuilt	1835.50	2087.50	0
5	Price	-419884.00	1500068.00	0

```
Outlier detection completed. Potential outliers are retained.
```

Next steps: Generate code with outlier_df | New interactive sheet

```
# Separate Features and Target
target_column = "Price"

# Id is an identifier, so it is not used as a predictive feature.
feature_columns = [
    "Area",
    "Bedrooms",
    "Bathrooms",
    "Floors",
    "YearBuilt",
    "Location",
    "Condition",
    "Garage"
]

X = df_clean[feature_columns].copy()
y = df_clean[target_column].copy()

numeric_features = [
    "Area", "Bedrooms", "Bathrooms", "Floors", "YearBuilt"
]

categorical_features = [
    "Location", "Condition", "Garage"
]

print("Features prepared successfully!")
print("Number of Features:", len(feature_columns))
print("Target Variable:", target_column)

print("\nNumerical Features:", numeric_features)
print("Categorical Features:", categorical_features)

print("\nFeature Data Shape:", X.shape)
print("Target Data Shape:", y.shape)
```

```
Features prepared successfully!
Number of Features: 8
Target Variable: Price

Numerical Features: ['Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt']
Categorical Features: ['Location', 'Condition', 'Garage']

Feature Data Shape: (2000, 8)
Target Data Shape: (2000,)
```

3. Conduct Exploratory Data Analysis (EDA) using descriptive statistics and visualizations to understand the characteristics ofthe dataset.

```
# Descriptive Statistical Analysis
print("Numerical Summary:")
display(df_clean.describe().T)

print("\nCategorical Summary:")
display(df_clean.describe(include="object").T)

print("\nAverage House Price:", round(df_clean["Price"].mean(), 2))
print("Median House Price:", round(df_clean["Price"].median(), 2))
print("Minimum House Price:", df_clean["Price"].min())
print("Maximum House Price:", df_clean["Price"].max())
```

Numerical Summary:

	count	mean	std	min	25%	50%	75%	max
Id	2000.0	1000.5000	577.494589	1.0	500.75	1000.5	1500.25	2000.0
Area	2000.0	2786.2095	1295.146799	501.0	1653.00	2833.0	3887.50	4999.0
Bedrooms	2000.0	3.0035	1.424606	1.0	2.00	3.0	4.00	5.0
Bathrooms	2000.0	2.5525	1.108990	1.0	2.00	3.0	4.00	4.0
Floors	2000.0	1.9935	0.809188	1.0	1.00	2.0	3.00	3.0
YearBuilt	2000.0	1961.4460	35.926695	1900.0	1930.00	1961.0	1993.00	2023.0
Price	2000.0	537676.8550	276428.845719	50005.0	300098.00	539254.0	780086.00	999656.0

Categorical Summary:

	count	unique	top	freq
Location	2000	4	Downtown	558
Condition	2000	4	Fair	521
Garage	2000	2	No	1038

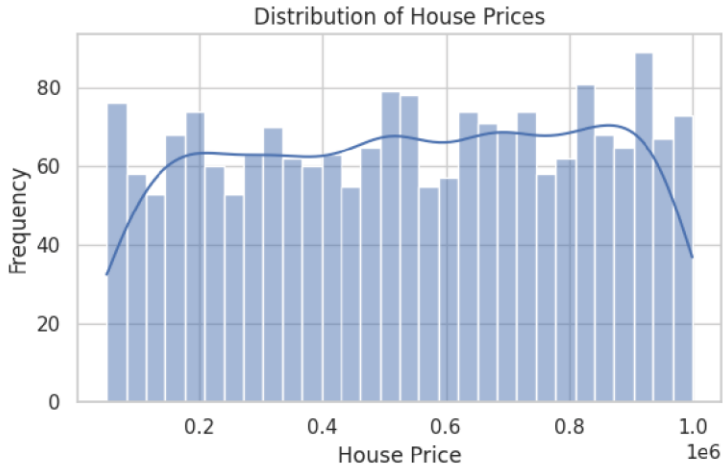
```
Average House Price: 537676.86
Median House Price: 539254.0
Minimum House Price: 50005
Maximum House Price: 999656
```

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

plt.figure(figsize=(6, 4))
sns.histplot(df_clean["Price"], bins=30, kde=True)

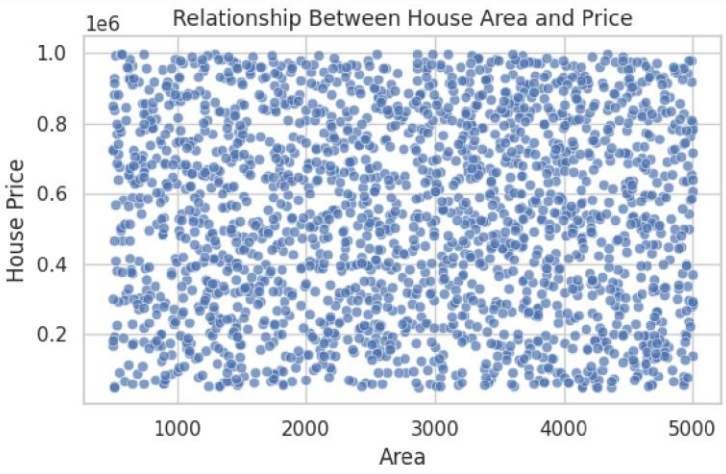
plt.title("Distribution of House Prices" )
plt.xlabel("House Price")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(6, 4))

sns.scatterplot(
    data=df_clean,
    x="Area",
    y="Price",
    alpha=0.7
)

plt.title("Relationship Between House Area and Price" )
plt.xlabel("Area")
plt.ylabel("House Price")
plt.tight_layout()
plt.show()
```



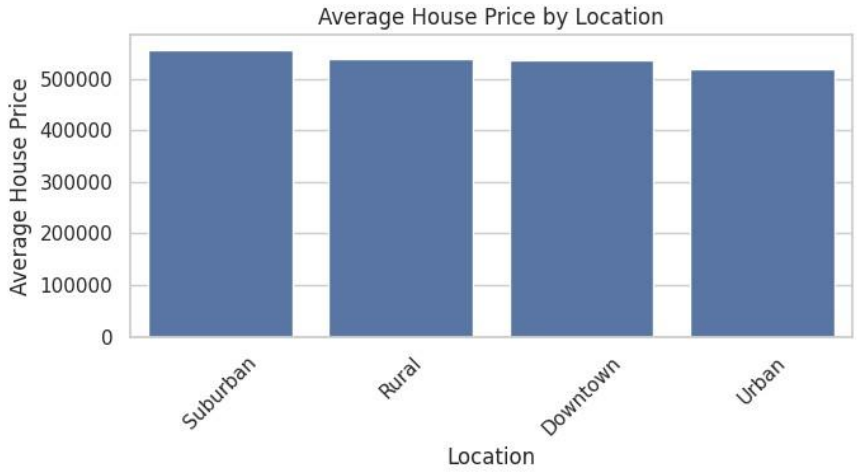

```
location_prices = (
    df_clean.groupby("Location")["Price"]
    .mean()
    .sort_values(ascending=False)
)

plt.figure(figsize=(7, 4))

sns.barplot(
    x=location_prices.index,
    y=location_prices.values
)

plt.title("Average House Price by Location" )
plt.xlabel("Location")
plt.ylabel("Average House Price" )
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
display(location_prices.rename("Average Price").to_frame())
```



Location	Average Price
Suburban	557416.333333
Rural	538613.943038
Downtown	536059.661290
Urban	518963.548454

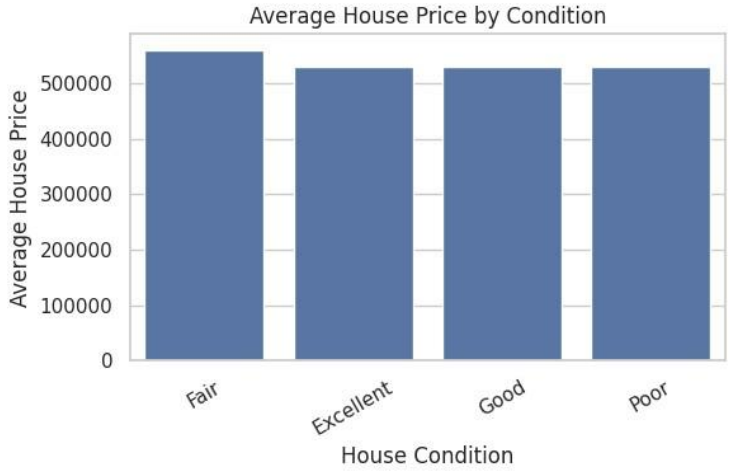
```
condition_prices = (
    df_clean.groupby("Condition")["Price"]
    .mean()
    .sort_values(ascending=False)
)

plt.figure(figsize=(6, 4))

sns.barplot(
    x=condition_prices.index ,
    y=condition_prices.values
)

plt.title("Average House Price by Condition" )
plt.xlabel("House Condition")
plt.ylabel("Average House Price")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

display(condition_prices.rename("Average Price").to_frame())
```



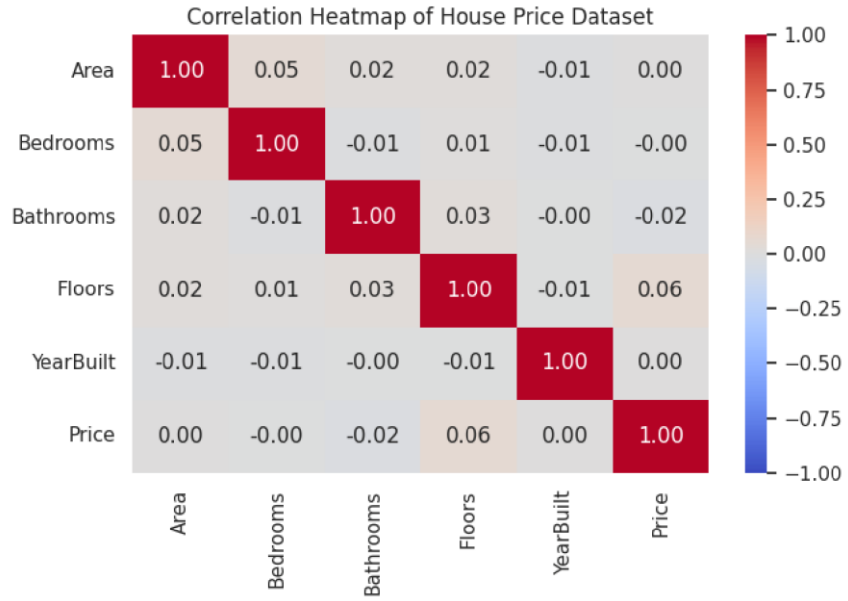
Condition	Average Price
Fair	560594.140115
Excellent	530812.009785
Good	529002.351410
Poor	528933.218935

```
# Correlation among numerical variables
correlation_matrix = df_clean[
    numeric_features + ["Price"]
].corr()
```

```
plt.figure(figsize=(7, 5))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    vmin=-1,
    vmax=1
)

plt.title("Correlation Heatmap of House Price Dataset" )
plt.tight_layout()
plt.show()
```



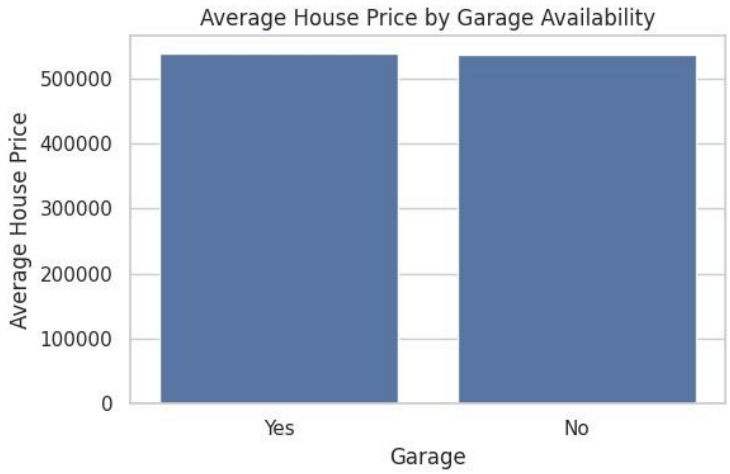

```
garage_prices = (
    df_clean.groupby("Garage")["Price"]
    .mean()
    .sort_values(ascending=False)
)

plt.figure(figsize=(6, 4))

sns.barplot(
    x=garage_prices.index,
    y=garage_prices.values
)

plt.title("Average House Price by Garage Availability" )
plt.xlabel("Garage")
plt.ylabel("Average House Price")
plt.tight_layout()
plt.show()

display(garage_prices.rename("Average Price").to_frame())
```



Garage	Average Price
Yes	538492.747401
No	536920.700385

4. Perform statistical analysis, including correlation analysis and appropriate hypothesis testing, to identify significantrelationships between variables.

```
from scipy.stats import pearsonr

# Correlation between Area and Price
correlation, p_value = pearsonr(
    df_clean["Area"],
    df_clean["Price"]
)

print("Correlation Analysis: Area vs Price")
print("Pearson Correlation Coefficient:", round(correlation, 4))
print("P-value:", p_value)

if p_value < 0.05:
    print("Result: The correlation is statistically significant.")
else:
    print("Result: The correlation is not statistically significant.")

if correlation > 0:
    print("Interpretation: Area and Price have a positive linear relationship.")
elif correlation < 0:
    print("Interpretation: Area and Price have a negative linear relationship.")
else:
    print("Interpretation: No linear correlation was detected.")
```

```
Correlation Analysis: Area vs Price
Pearson Correlation Coefficient: 0.0015
P-value: 0.9450511952634
Result: The correlation is not statistically significant.
Interpretation: Area and Price have a positive linear relationship.
```

```
from scipy.stats import f_oneway

# Prepare price groups for each location
location_groups = [
    group["Price"].values
    for _, group in df_clean.groupby("Location")
    if len(group) >= 2
]

print("Number of Location Groups:", len(location_groups))

if len(location_groups) >= 2:
    f_statistic, anova_p_value = f_oneway(*location_groups)

    print("ANOVA F-statistic:", round(f_statistic, 4))
    print("ANOVA P-value:", anova_p_value)

    if anova_p_value < 0.05:
        print("Decision: Reject the null hypothesis.")
        print("Conclusion: At least one location has a significantly different mean price.")
    else:
        print("Decision: Fail to reject the null hypothesis.")
        print("Conclusion: No statistically significant difference was detected.")
else:
    print("ANOVA could not be performed because fewer than two valid groups exist.")
```

```
Number of Location Groups: 4
ANOVA F-statistic: 1.5714
ANOVA P-value: 0.1943220538614731
Decision: Fail to reject the null hypothesis.
Conclusion: No statistically significant difference was detected.
```

5. Create informative visualizations and dashboards using Python libraries or Business Intelligence tools such as Tableau orMicrosoft Power BI.

```
import plotly.express as px
from IPython.display import display

# 1. House Price Distribution
fig1 = px.histogram(
    df_clean,
    x="Price",
    nbins=30,
    title="House Price Distribution",
    marginal="box"
)

fig1.show()

# 2. Area vs Price
fig2 = px.scatter(
    df_clean,
    x="Area",
    y="Price",
    color="Location",
    hover_data=["Bedrooms", "Bathrooms", "Condition"],
    title="House Area vs Price"
```

```
)

fig2.show()

# 3. Average Price by Location
location_summary = (
    df_clean.groupby("Location", as_index=False)
    .agg(Average_Price=("Price", "mean"))
)

fig3 = px.bar(
    location_summary,
    x="Location",
    y="Average_Price",
    title="Average House Price by Location",
    color="Location"
)

fig3.show()

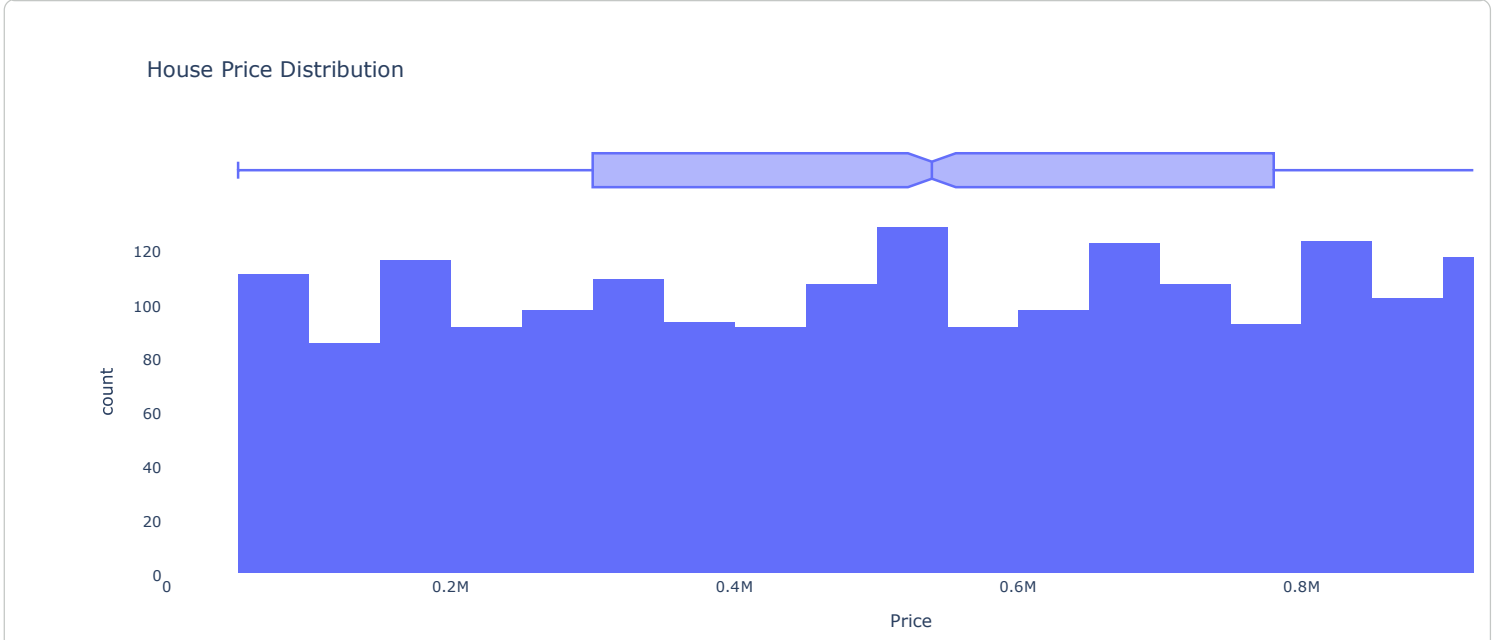
# 4. Average Price by Condition
condition_summary = (
    df_clean.groupby("Condition", as_index=False)
    .agg(Average_Price=("Price", "mean"))
)

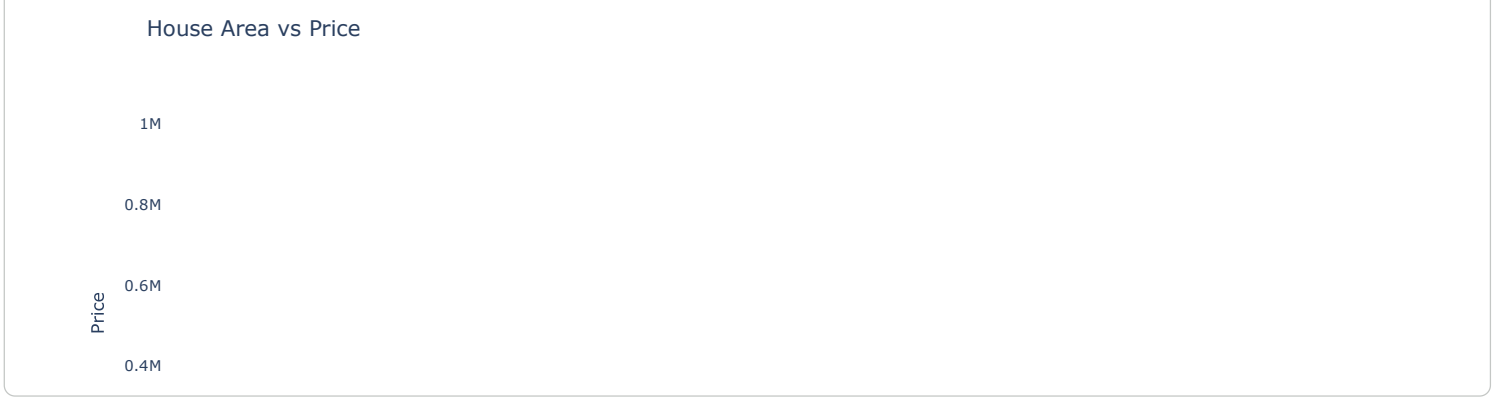
fig4 = px.bar(
    condition_summary,
    x="Condition",
    y="Average_Price",
    title="Average House Price by Condition",
    color="Condition"
)

fig4.show()

# 5. Correlation Heatmap
fig5 = px.imshow(
    correlation_matrix,
    text_auto=".2f",
    aspect="auto",
    color_continuous_scale="RdBu_r",
    zmin=-1,
    zmax=1,
    title="Correlation Heatmap"
)

fig5.show()
```





6. Build an appropriate machine learning model (e.g., Linear Regression, Decision Tree, or K-Means Clustering) based on the problem statement.

```
from sklearn.model_selection  import train_test_split

X = df_clean[feature_columns].copy()
y = df_clean["Price"].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)

print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

```
Training Data Shape: (1600, 8)
Testing Data Shape: (400, 8)

Training Samples: 1600
Testing Samples: 400
```

```
from sklearn.compose  import ColumnTransformer
from sklearn.pipeline  import Pipeline
from sklearn.impute  import SimpleImputer
from sklearn.preprocessing  import OneHotEncoder
from sklearn.linear_model  import LinearRegression

# Numerical preprocessing
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median"))
])

# Categorical preprocessing
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

# Combine numerical and categorical preprocessing
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

# Complete machine learning pipeline
model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

# Train the model
model.fit(X_train, y_train)

print("Linear Regression Model Trained Successfully!")
```

```
Linear Regression Model Trained Successfully!
```

```
# Predict house prices on unseen test data
y_pred = model.predict(X_test)

results_df = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})

results_df["Prediction Error"] = (
    results_df["Actual Price"] - results_df["Predicted Price"]
)

print("Actual vs Predicted House Prices:")
display(results_df.head(10))
```

```
Actual vs Predicted House Prices:
```

	Actual Price	Predicted Price	Prediction Error
0	514764	521988.221898	-7224.221898
1	694256	549119.311967	145136.688033
2	66375	487101.222356	-420726.222356
3	650243	539752.743993	110490.256007
4	223285	553242.248725	-329957.248725
5	468127	521375.920258	-53248.920258
6	513002	523320.180806	-10318.180806
7	911525	578133.643533	333391.356467
8	723265	545899.647385	177365.352615
9	339416	577368.699406	-237952.699406



7. Evaluate the model using suitable performance metrics such as Accuracy, Precision, Recall, F1-Score, MAE, RMSE, or R^2 Score,depending on the selected algorithm.

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Model Evaluation Results:\n")
print("Mean Absolute Error (MAE):", round(mae, 2))
print("Mean Squared Error (MSE):", round(mse, 2))
print("Root Mean Squared Error (RMSE):", round(rmse, 2))
print("R² Score:", round(r2, 4))
print("R² Score (%):", round(r2 * 100, 2), "%")
```

```
Model Evaluation Results:

Mean Absolute Error (MAE): 243241.98
Mean Squared Error (MSE): 78321466146.03
Root Mean Squared Error (RMSE): 279859.73
R² Score: -0.0067
R² Score (%): -0.67 %
```

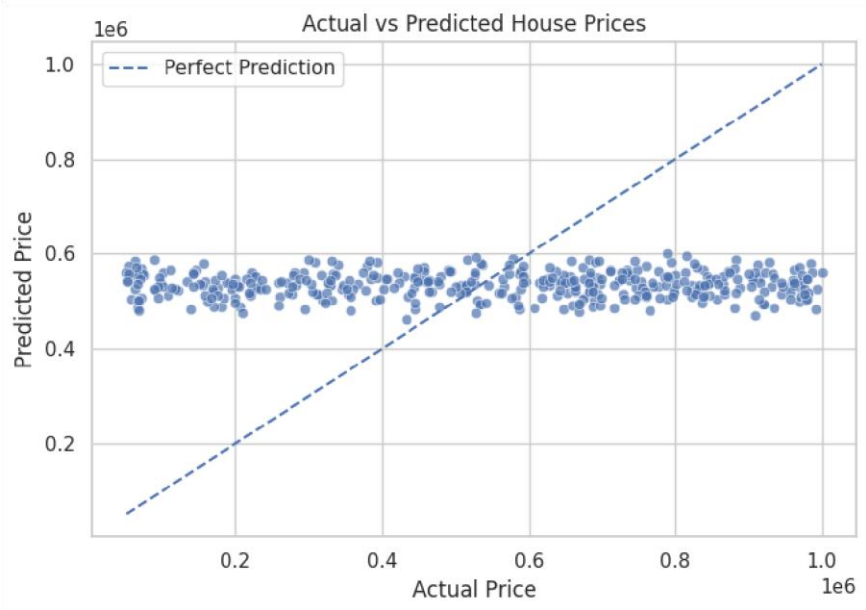
```
plt.figure(figsize=(7, 5))

sns.scatterplot(
    x=y_test,
    y=y_pred,
    alpha=0.7
)
```

```
# Reference line for perfect predictions
min_value = min(y_test.min(), y_pred.min())
max_value = max(y_test.max(), y_pred.max())

plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    linestyle="--",
    label="Perfect Prediction"
)

plt.title("Actual vs Predicted House Prices" )
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.legend()
plt.tight_layout()
plt.show()
```




```
residuals = y_test.values - y_pred

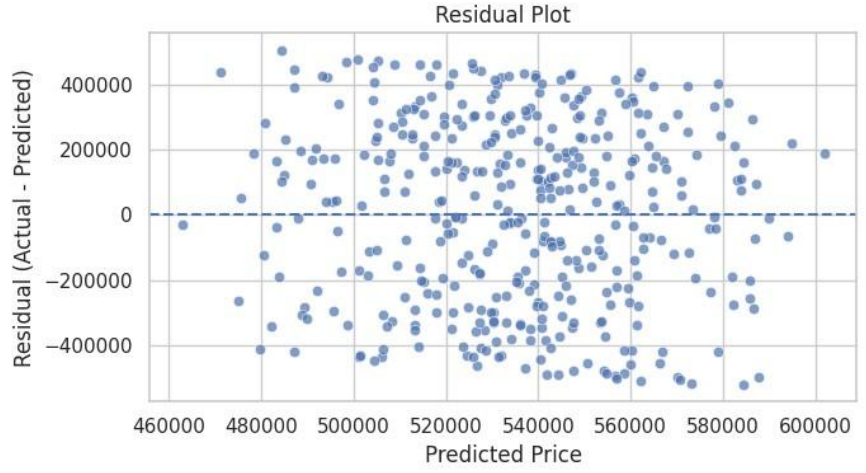
plt.figure(figsize=(7, 4))

sns.scatterplot(
    x=y_pred,
    y=residuals,
    alpha=0.7
)

plt.axhline(
    y=0,
    linestyle="--"
)

plt.title("Residual Plot")
plt.xlabel("Predicted Price")
plt.ylabel("Residual (Actual - Predicted)" )
plt.tight_layout()
plt.show()

print("Mean Residual:", round(np.mean(residuals), 2))
```




```
Mean Residual: 8623.22
```

```
train_predictions = model.predict (X_train)
```

```
train_r2 = r2_score(y_train, train_predictions)
test_r2 = r2_score(y_test, y_pred)

print("Training R² Score:", round(train_r2, 4))
print("Testing R² Score:", round(test_r2, 4))

if train_r2 - test_r2 > 0.15:
    print("The model shows a noticeable training-testing per formance gap.")
else:
    print("No large R² performance gap was detected using  the 0.15 threshold.")

print("Note: This is a diagnostic check, not definitive  proof of overfitting.")
```

```
Training R² Score: 0.0099
Testing R² Score: -0.0067
No large R² performance gap was detected using the 0.15 threshold.
Note: This is a diagnostic check, not definitive proof of overfitting.
```

8. Interpret the analytical results and identify key business insights, trends, opportunities, or potential risks.

```
# Overall Business Insights
average_price = df_clean["Price"].mean()
median_price = df_clean["Price"].median()

highest_price_house = df_clean.loc[df_clean["Price"].idxmax()]
lowest_price_house = df_clean.loc[df_clean["Price"].idxmin()]

print("BUSINESS INSIGHTS:\n")
print("Total Houses Analyzed:", len(df_clean))
print("Average House Price:", round(average_price, 2))
print("Median House Price:", round(median_price, 2))

print("\nHighest-Priced House:")
print(highest_price_house[
    ["Area", "Bedrooms", "Location", "Condition", "Price"]
])

print("\nLowest-Priced House:")
print(lowest_price_house[
    ["Area", "Bedrooms", "Location", "Condition", "Price"]
])

# Location with highest average price
if not location_prices.empty:
    print("\nLocation with Highest Average Price:",
          location_prices.idxmax())
    print("Average Price:",
          round(location_prices.max(), 2))

# Location with lowest average price
if not location_prices.empty:
    print("\nLocation with Lowest Average Price:",
          location_prices.idxmin())
    print("Average Price:",
          round(location_prices.min(), 2))

# Condition with highest average price
if not condition_prices.empty:
    print("\nCondition with Highest Average Price:",
          condition_prices.idxmax())
    print("Average Price:",
          round(condition_prices.max(), 2))
```

```
BUSINESS INSIGHTS:

Total Houses Analyzed: 2000
Average House Price: 537676.86
Median House Price: 539254.0

Highest-Priced House:
Area             3099
Bedrooms            1
Location     Suburban
Condition        Good
Price          999656
Name: 1004, dtype: object

Lowest-Priced House:
Area          4260
Bedrooms         2
Location     Rural
Condition     Fair
Price        50005
Name: 1167, dtype: object

Location with Highest Average Price: Suburban
Average Price: 557416.33

Location with Lowest Average Price: Urban
Average Price: 518963.55

Condition with Highest Average Price: Fair
Average Price: 560594.14
```

```
from sklearn.inspection import permutation_importance

importance_result = permutation_importance(
    model,
    X_test,
    y_test,
    scoring="r2",
    n_repeats=10,
    random_state=42,
    n_jobs=-1
)

importance_df = pd.DataFrame({
    "Feature": feature_columns,
    "Importance": importance_result.importances_mean,
    "Importance Std": importance_result.importances_std
}).sort_values("Importance", ascending=False)

print("Feature Importance Results:")
display(importance_df)

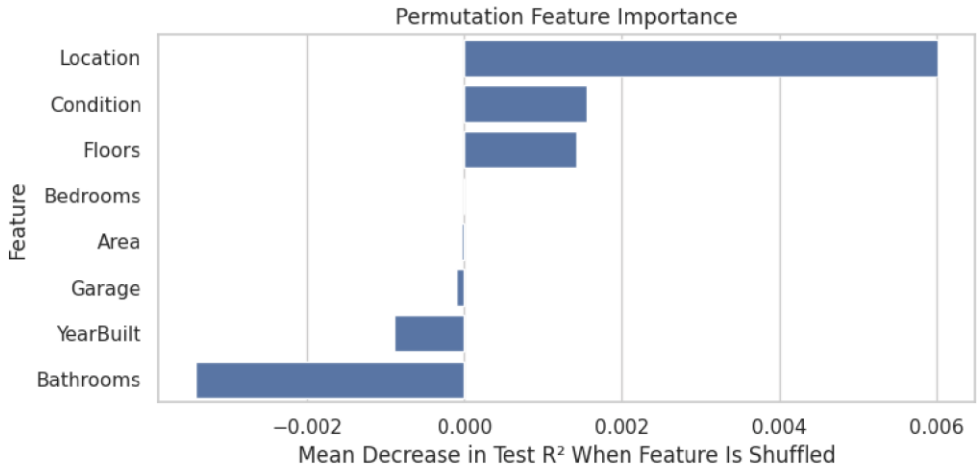
plt.figure(figsize=(8, 4))

sns.barplot(
    data=importance_df,
    x="Importance",
    y="Feature"
)

plt.title("Permutation Feature Importance")
plt.xlabel("Mean Decrease in Test R² When Feature Is Shuffled")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

```
Feature Importance Results:
```

	Feature	Importance	Importance Std
5	Location	0.006016	0.002505
6	Condition	0.001554	0.004787
3	Floors	0.001423	0.006138
1	Bedrooms	-0.000008	0.000049
0	Area	-0.000035	0.000247
7	Garage	-0.000095	0.000348
4	YearBuilt	-0.000892	0.001293
2	Bathrooms	-0.003416	0.003663



Next steps: Generate code with `importance_df` New interactive sheet

BUSINESS RECOMMENDATIONS

1. Pricing Strategy: Use the model's predicted prices as an initial reference for property valuation.
2. Location Analysis: Compare average house prices across locations to identify higher- and lower-priced markets.
3. Property Features: The most influential feature in this model is: Location Study this feature along with location and property condition when assessing prices.
4. Model Evaluation: Review MAE and RMSE to understand the typical scale of prediction errors.
5. Practical Limitation: Validate predictions against current local market prices before making investment decisions.

9. **Prepare a professional project report and presentation summarizing the methodology, findings, visualizations, modelperformance, and recommendations.**

```
from datetime import datetime
```

```
report = f"""
EXPERIMENT NO. 10
END-TO-END DATA ANALYTICS PROJECT:
BUSINESS INTELLIGENCE AND PREDICTIVE ANALYTICS

1. AIM
To implement a complete data analytics workflow using a real-world
House Price Prediction dataset.

2. DATASET
Dataset Name: House Price Prediction Dataset
Original File: {file_name}
Original Rows: {df.shape[0]}
Original Columns: {df.shape[1]}
Rows After Cleaning: {df_clean.shape[0]}
Target Variable: Price

3. METHODOLOGY
- Dataset collection and loading
- Data cleaning and preprocessing
- Duplicate and missing-value checks
- IQR-based outlier detection
- Exploratory Data Analysis
- Statistical correlation analysis
- One-way ANOVA hypothesis testing
- Data visualization and interactive charts
- Linear Regression model training
- Model evaluation
- Business insight generation

4. EXPLORATORY DATA ANALYSIS
Average House Price: {average_price:.2f}
Median House Price: {median_price:.2f}
Minimum House Price: {df_clean['Price'].min():.2f}
Maximum House Price: {df_clean['Price'].max():.2f}

5. STATISTICAL ANALYSIS
Pearson Correlation (Area vs Price): {correlation:.4f}
Correlation P-value: {p_value:.6g}
ANOVA F-statistic: {f_statistic if 'f_statistic' in globals() else float('nan'):.4f}
ANOVA P-value: {anova_p_value if 'anova_p_value' in globals() else float('nan'):.6g}

6. MACHINE LEARNING MODEL
Algorithm: Linear Regression
Training Samples: {len(X_train)}
Testing Samples: {len(X_test)}
Input Features: {', '.join(feature_columns)}
Target Variable: Price

7. MODEL PERFORMANCE
Mean Absolute Error (MAE): {mae:.2f}
Mean Squared Error (MSE): {mse:.2f}
Root Mean Squared Error (RMSE): {rmse:.2f}
R2 Score: {r2:.4f}

8. IMPORTANT FEATURES
{importance_df[['Feature', 'Importance']].to_string(index=False)}

9. BUSINESS RECOMMENDATIONS
- Use predicted prices as an initial property valuation reference.
- Compare prices across locations and property conditions.
- Evaluate prediction errors before using the model operationally.
- Validate model estimates using current local market data.
- Retrain and reevaluate the model when new data becomes available.

10. CONCLUSION
This project demonstrates an end-to-end analytics workflow involving
data preprocessing, exploratory analysis, statistical testing,
visualization, regression modeling and business recommendations.

The model's real-world usefulness depends on the measured test-set
performance, data quality and whether the dataset represents the
current housing market.

Report Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}
"""

with open("House_Price_Analytics_Report.txt", "w", encoding="utf-8") as file:
    file.write(report)

print(report)
print("\nReport saved as House_Price_Analytics_Report.txt")
```


```
EXPERIMENT NO. 10
END-TO-END DATA ANALYTICS PROJECT:
BUSINESS INTELLIGENCE AND PREDICTIVE ANALYTICS

1. AIM
To implement a complete data analytics workflow using a real-world
House Price Prediction dataset.

2. DATASET
Dataset Name: House Price Prediction Dataset
```

```
Original File: House Price Prediction Dataset.csv
Original Rows: 2000
Original Columns: 10
Rows After Cleaning: 2000
Target Variable: Price

3. METHODOLOGY
- Dataset collection and loading
- Data cleaning and preprocessing
- Duplicate and missing-value checks
- IQR-based outlier detection
- Exploratory Data Analysis
- Statistical correlation analysis
- One-way ANOVA hypothesis testing
- Data visualization and interactive charts
- Linear Regression model training
- Model evaluation
- Business insight generation

4. EXPLORATORY DATA ANALYSIS
Average House Price: 537676.85
Median House Price: 539254.00
Minimum House Price: 50005.00
Maximum House Price: 999656.00

5. STATISTICAL ANALYSIS
Pearson Correlation (Area vs Price): 0.0015
Correlation P-value: 0.945051
ANOVA F-statistic: 1.5714
ANOVA P-value: 0.194322

6. MACHINE LEARNING MODEL
Algorithm: Linear Regression
Training Samples: 1600
Testing Samples: 400
Input Features: Area, Bedrooms, Bathrooms, Floors, YearBuilt, Location, Condition, Garage
Target Variable: Price

7. MODEL PERFORMANCE
Mean Absolute Error (MAE): 243241.98
Mean Squared Error (MSE): 78321466146.03
Root Mean Squared Error (RMSE): 279859.73
R2 Score: -0.0067

8. IMPORTANT FEATURES
  Feature  Importance
 Location    0.006016
```

```
# Save Actual vs Predicted Results
results_df.to_csv(
    "House_Price_Predictions.csv",
    index=False
)

# Save Feature Importance Results
importance_df.to_csv(
    "House_Price_Feature_Importance.csv",
    index=False
)

# Save cleaned dataset
df_clean.to_csv(
    "House_Price_Cleaned_Dataset.csv",
    index=False
)

print("Files exported successfully:")
print("1. House_Price_Analytics_Report.txt")
print("2. House_Price_Predictions.csv")
print("3. House_Price_Feature_Importance.csv")
print("4. House_Price_Cleaned_Dataset.csv")
```

```
Files exported successfully:
1. House_Price_Analytics_Report.txt
2. House_Price_Predictions.csv
3. House_Price_Feature_Importance.csv
4. House_Price_Cleaned_Dataset.csv
```

```
from google.colab import files

files.download("House_Price_Analytics_Report.txt")
```

```
files.download("House_Price_Predictions.csv")
files.download("House_Price_Feature_Importance.csv")
files.download("House_Price_Cleaned_Dataset.csv")
```

10. Present the complete analytics solution and discuss how the results can support business strategy and decision-making.

```
print("HOUSE PRICE PREDICTION: DATA ANALYTICS PROJECT:")

print("\n1. BUSINESS PROBLEM")
print("Estimate house prices using property characterist ics.")
```

```
print("\n2. DATASET")
print("Total Records Analyzed:", len(df_clean))
print("Input Features:", len(feature_columns))
print("Prediction Target: Price")

print("\n3. ANALYTICAL APPROACH")
print("Data Cleaning -> EDA -> Statistical Analysis")
print("-> Visualization -> Linear Regression -> Evaluation")

print("\n4. KEY FINDINGS")
print("Average House Price:", round(average_price, 2))
print("Median House Price:", round(median_price, 2))
print("Area-Price Correlation:", round(correlation, 4))

print("\n5. MODEL PERFORMANCE")
print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))
print("R² Score:", round(r2, 4))

print("\n6. MOST IMPORTANT MODEL FEATURE")
print(importance_df.iloc[0]["Feature"])

print("\n7. BUSINESS VALUE")
print("The model can help estimate property values,")
print("compare market segments, and support initial pricing decisions.")

print("\n8. LIMITATIONS")
print("Predictions depend on dataset quality and representativeness.")
print("The model should be validated against current market data.")
print("Statistical associations do not necessarily imply causation.")

print("\n9. CONCLUSION")
print("An end-to-end data analytics workflow was implemented")
print("using descriptive analysis, statistical testing,")
print("visualizations, predictive modeling, and evaluation.")
print("\nPROJECT PRESENTATION SUMMARY COMPLETED")
```

```
HOUSE PRICE PREDICTION: DATA ANALYTICS PROJECT:

1. BUSINESS PROBLEM
Estimate house prices using property characteristics.

2. DATASET
Total Records Analyzed: 2000
Input Features: 8
Prediction Target: Price

3. ANALYTICAL APPROACH
Data Cleaning -> EDA -> Statistical Analysis
-> Visualization -> Linear Regression -> Evaluation

4. KEY FINDINGS
Average House Price: 537676.86
Median House Price: 539254.0
Area-Price Correlation: 0.0015

5. MODEL PERFORMANCE
```

Questions and Answers

1. What are the major stages involved in an end-to-end data analytics project?

The major stages are data collection, data preprocessing, Exploratory Data Analysis (EDA), statistical analysis, data visualization, machine learning model development, model evaluation, interpretation of results, and presentation of business insights.

2. Why is data preprocessing considered the foundation of successful analytics and machine learning?

Data preprocessing improves data quality by handling missing values, duplicate records, outliers, and categorical variables. Clean and properly prepared data helps machine learning models produce more reliable results and reduces errors caused by poor-quality data.

3. How does Exploratory Data Analysis (EDA) contribute to model development?

EDA helps understand the dataset through descriptive statistics and visualizations. It identifies patterns, distributions, correlations, and unusual values. These findings help select relevant features, choose suitable algorithms, and make better modeling decisions.

4. What factors should be considered while selecting a machine learning algorithm for a business problem?

The main factors are the type of problem, dataset size, data characteristics, required accuracy, interpretability, computational resources, training time, and evaluation metrics. For example, Linear Regression is suitable for predicting numerical values, while Decision Trees can be used for classification and regression.

5. Why is model evaluation essential before deploying a predictive model?

Model evaluation measures how well a model performs on unseen data. It helps identify prediction errors, overfitting, and reliability issues. Suitable metrics, such as MAE, RMSE, and R^2 for regression or Accuracy, Precision, Recall, and F1-Score for classification, help determine whether a model is suitable for practical use.

6. Explain how Business Intelligence tools complement machine learning in data analytics projects.

Business Intelligence tools such as Microsoft Power BI and Tableau convert analytical results into interactive dashboards, charts, and reports. Machine learning makes predictions and identifies patterns, while BI tools help communicate these results and support business decision-making.

7. What challenges are commonly encountered while working with real-world datasets?

Common challenges include missing values, duplicate records, outliers, inconsistent formats, incorrect data, class imbalance, insufficient data, privacy concerns, and computational limitations. These issues must be addressed to obtain reliable analytical results.

8. How can analytical insights be converted into actionable business recommendations?

Analytical insights can be converted into recommendations by identifying important trends, understanding business problems, setting measurable objectives, and proposing specific actions. For example, house price analysis can help businesses estimate property values, compare locations, and develop pricing strategies. Recommendations should be tested and monitored using measurable outcomes.

9. Why is effective presentation and data storytelling important in analytics projects?

Effective presentation and data storytelling make complex analytical results easier to understand. Charts, dashboards, and clear explanations help stakeholders identify important findings, understand their business impact, and make informed decisions.

10. Suggest future improvements or advanced techniques that could enhance the accuracy and effectiveness of the developed analytics solution.

The solution can be improved by collecting more recent and representative data, performing advanced feature engineering, tuning model hyperparameters, and comparing Linear Regression with models such as Random Forest, Gradient Boosting, and XGBoost. Cross-validation, appropriate outlier handling, and additional evaluation metrics can improve reliability. Deploying the model through a web application and monitoring its performance over time can further enhance its practical usefulness.